# Where the Headroom Lies: Block $\ell_p$-Norm Statistics for Cyclic Join Bounds

Extended version: full proofs and additional experiments

Suan Lee
Semyung University, Jecheon, Republic of Korea
suanlee@semyung.ac.kr

Jinho Kim
Kangwon National University, Chuncheon, Republic of Korea
jhkim@kangwon.ac.kr

This note accompanies the ICDE 2027 submission and gives the proofs that the paper states in condensed form: the $\ell_p$ Composition Inequality (the validity of every LpH-CEG bound) and the Cyclic Improvement theorem, and the $bs = 1$ experiments on JOB, a TPC-H-like schema, data size and skew. Notation follows the paper.

## 1 The $\ell_p$ Composition Inequality

**Setup.** A hyperpath $P = (S_0{=}\emptyset, F_1, S_1, \ldots, F_m, S_m)$ adds a block of relations $F_j$ at step $j$, $S_j = S_{j-1} \cup F_j$, and $S_m$ is the set of all relations. Write $A_j = \mathrm{attr}(F_j)$, $V_j = \mathrm{attr}(S_j)$ ($V_0 = \emptyset$, $V_m = V$), the interface $I_j = A_j \cap V_{j-1}$ and $E_j = A_j \setminus I_j$. Then

(F1) $E_j = V_j \setminus V_{j-1}$, so $\{E_j\}_j$ partitions $V$;

(F2) $I_j \subseteq V_{j-1}$.

Weights $w_{j,p} \geq 0$ ($p \in \mathcal{P} \subseteq [1, \infty]$) satisfy the attribute-wise cover constraint of the paper: for every attribute $a \in V$, bound at step $j(a)$ (i.e. $a \in E_{j(a)}$),

$$w_{j(a)} + \sum_{j' :\, a \in I_{j'}} s_{j'} \;\geq\; 1, \qquad w_j := \sum_p w_{j,p}, \quad s_j := \sum_p w_{j,p}/p \;\; (1/\infty = 0). \tag{C}$$

A *polymatroid* is $h : 2^V \to \mathbb{R}$ with $h(\emptyset) = 0$, monotone and submodular; $h(X \mid Y) := h(X \cup Y) - h(Y)$.

**Lemma 1** (Conditioning)**.** *For a polymatroid $h$ and $Y \subseteq Z$: $h(X \mid Y) \geq h(X \mid Z)$.*

*Proof.* Submodularity on $X \cup Y$ and $Z$ gives $h(X \cup Y) + h(Z) \geq h(X \cup Y \cup Z) + h((X \cup Y) \cap Z)$, and $h((X \cup Y) \cap Z) \geq h(Y)$ by monotonicity since $Y \subseteq (X \cup Y) \cap Z$. Rearrange. □

**Lemma 2** (Fractional subadditivity)**.** *Let $f$ be monotone submodular on subsets of a finite set with $f(\emptyset) = 0$, and let $\lambda_i \geq 0$ satisfy $\sum_{i:\, a \in T_i} \lambda_i \geq 1$ for every $a \in S$. Then $\sum_i \lambda_i f(T_i) \geq f(S)$.*

*Proof.* Order $S = (a_1, \ldots, a_k)$ and let $S_{<t} = \{a_1, \ldots, a_{t-1}\}$. By monotonicity, the chain rule and Lemma 1, $f(T_i) \geq f(T_i \cap S) = \sum_{a_t \in T_i} f(a_t \mid T_i \cap S_{<t}) \geq \sum_{a_t \in T_i} f(a_t \mid S_{<t})$. Weighting by $\lambda_i$ and exchanging sums, $\sum_i \lambda_i f(T_i) \geq \sum_t f(a_t \mid S_{<t}) \sum_{i:\, a_t \in T_i} \lambda_i \geq \sum_t f(a_t \mid S_{<t}) = f(S)$, using $f(a_t \mid S_{<t}) \geq 0$. □

**Lemma 3** (The obligation)**.** *For every polymatroid $h$ and every $w$ satisfying (C),*

$$\sum_j \Big[ s_j \, h(I_j) + w_j \, h(E_j \mid I_j) \Big] \;\geq\; h(V).$$

*Proof.* Let $g_j = h(E_j \mid V_{j-1})$, $\delta_j = \max(0, 1 - w_j) \in [0, 1]$ and $f_j(S) = h(S \mid V_{j-1})$ for $S \subseteq E_j$; $f_j$ is monotone, submodular and $f_j(\emptyset) = 0$.

(i) For each $j'$, the chain rule along the partition (F1) and Lemma 1 give $h(I_{j'}) = \sum_j h(I_{j'} \cap E_j \mid I_{j'} \cap V_{j-1}) \geq \sum_j f_j(I_{j'} \cap E_j)$. Hence $\sum_{j'} s_{j'} h(I_{j'}) \geq \sum_j \sum_{j'} s_{j'} f_j(I_{j'} \cap E_j)$.

(ii) Fix $j$. If $\delta_j > 0$ then by (C) every $a \in E_j$ has $\sum_{j' : a \in I_{j'}} s_{j'} \geq 1 - w_j = \delta_j$, so the sets $I_{j'} \cap E_j$ with weights $s_{j'}/\delta_j$ fractionally cover $E_j$, and Lemma 2 gives $\sum_{j'} s_{j'} f_j(I_{j'} \cap E_j) \geq \delta_j \, g_j$. If $\delta_j = 0$ the same inequality holds because the left side is non-negative.

(iii) By (F2) and Lemma 1, $h(E_j \mid I_j) \geq g_j \geq 0$, and $w_j \geq 1 - \delta_j$, so $w_j h(E_j \mid I_j) \geq (1 - \delta_j) g_j$.

Adding (i)–(iii): the left side is at least $\sum_j [\delta_j + (1 - \delta_j)] g_j = \sum_j h(E_j \mid V_{j-1}) = h(V)$ by the chain rule and (F1). □

**Theorem 1** ($\ell_p$ Composition Inequality)**.** *Let $D$ be a database, $\mathcal{Q}(D) \neq \emptyset$, and let $\mu_p(F_j \mid I_j)$ be the $\ell_p$-norm of the degree sequence of $E_j$-values per $I_j$-value in a relation $R_j \supseteq \pi_{A_j} \mathcal{Q}(D)$—for instance the block join $\bowtie_{R \in F_j} R$ computed exactly on $D$. If $w$ satisfies (C), then*

$$|\mathcal{Q}(D)| \;\leq\; \prod_{j=1}^{m} \prod_{p \in \mathcal{P}} \mu_p(F_j \mid I_j)^{\, w_{j,p}}.$$

*Proof.* Let $h$ be the entropy of the uniform distribution on $\mathcal{Q}(D)$; it is a polymatroid and $h(V) = \log_2 |\mathcal{Q}(D)|$. The marginal on $A_j$ is supported on $\pi_{A_j} \mathcal{Q}(D) \subseteq R_j$, so the $\ell_p$ degree inequality of Abo Khamis et al. (PODS 2024) gives, for $p \in [1, \infty]$, $\frac{1}{p} h(I_j) + h(E_j \mid I_j) \leq \log_2 \mu_p(F_j \mid I_j)$. Multiply by $w_{j,p} \geq 0$, sum, and apply Lemma 3. □

**Scope.** The proof uses only monotonicity and submodularity, so it holds over the whole polymatroid cone and for every hyperpath, block size and $p \geq 1$. It requires (C) exactly: an LP solver may return weights violating (C) by about $10^{-7}$; scaling $w$ by $1/(1 - \varepsilon)$, for a coverage shortfall $\varepsilon = 1 - \min_a$(coverage of $a$), restores (C) and raises a bound $B$ by the factor $B^{\varepsilon/(1-\varepsilon)}$, which grows with $B$. At the solver's feasibility tolerance ($\varepsilon \leq 10^{-7}$) and the largest cover bound we report ($B < 2^{100}$) the factor is below $1 + 10^{-5}$; the released code applies it; the bounds reported in the paper were computed before it did and sit below their rescaled values by at most that factor. It requires exact statistics of the database being bounded (or deterministic upper bounds on them, since the bound is monotone in each $\mu$); a sampled or sketched statistic, such as the Hutchinson estimate of the common-neighbour $\ell_2$ in the paper, is outside the guarantee. When some block join is empty the query is empty and the inequality holds trivially (the released LP gives such a factor weight zero, which only loosens the bound).

**Proposition 1** (Necessity)**.** *If nonnegative weights $(w_{j,p})$ make the product above an upper bound on $|\mathcal{Q}(D)|$ for every database $D$ whose atoms' relations are independent (no two atoms are copies of one relation), they satisfy (C). Hence on a fixed hyperpath the cover LP is optimal over all weightings of its statistics that are valid on every such database.*

*Proof.* Fix $a \in V$ and $N \geq 2$, and let $D_N$ give $a$ the domain $[N]$ and every other attribute a single value, each relation holding all tuples over these domains. Then $|\mathcal{Q}(D_N)| = N$, and the block join of $F_j$ has $N$ tuples if $a \in A_j$ and one otherwise, so $\mu_p(F_j \mid I_j)$ is $N^{1/p}$ if $a \in I_j$, $N$ if $a \in E_j$, and 1 if $a \notin A_j$. The bound is $N^{c_a}$ with $c_a$ the left side of (C) at $a$, and $N^{c_a} \geq N$ forces $c_a \geq 1$. (The entropy of $D_N$'s join is the modular polymatroid $h(S) = [a \in S] \log_2 N$, so the proposition also holds over the polymatroid cone.) □

**Self-joins.** The construction sets each atom's relation independently, which a self-join forbids: in $R(a,b) \bowtie R(b,c) \bowtie R(c,a)$ the three atoms share $R$, $D_N$ is not a database of that query, and indeed weight 3 on the first step's $|R|$ and 0 elsewhere gives the valid bound $|R|^3$ although $c$ is uncovered. Sufficiency (the Composition Inequality) is unaffected, since every self-join database is also a database of the independent query; necessity, and with it the reading that reweighting a fixed hyperpath's statistics is exhausted, holds for independent atoms only.

**Why the elementary argument does not suffice.** If $w_j \geq 1$ for every step, Lemma 3 follows from (iii) alone. The LP's optima on cyclic queries have $w_j < 1$ (e.g. $1/3$ on the closing step of a cycle); the deficit $\delta_j$ is paid for by later interfaces, which is exactly step (ii). The cyclic improvement and the need for Lemma 2 have the same source.

**Independent checks.** An LP over the polymatroid cone decides Lemma 3 at fixed weights; enumerating every vertex of the feasible weight region decides it for all weights. Over 6,197 hyperpath structures on eleven shapes with $|V| \leq 6$ and block sizes 1–3 the minimum slack is 0 to solver precision; an adversarial search over random abstract hyperpath schemes with $|V| \leq 5$ found no violation; and over 207,000 database–block-size checks the bound never fell below the exact count (`scripts/shannon_check_wide.py`, `scripts/validity_search.py`).

## 2 The Cyclic Improvement theorem

Order the cycle $C_k$ as $R_1(a_1, a_2), \ldots, R_k(a_k, a_1)$ and extend in this order with single-relation blocks. Let $N_j = |R_j|$, $M_j$ the maximum degree and $\mu_p(R_j)$ the $\ell_p$-norm of the degree sequence of $R_j$ from $a_j$ to $a_{j+1}$.

**Structure.** Step 1 has interface $\emptyset$ and extension $\{a_1, a_2\}$: its degree sequence is the single value $N_1$. A middle step $2 \leq j \leq k-1$ has interface $\{a_j\}$, extension $\{a_{j+1}\}$ and $\mu_\infty = M_j$. The closing step $k$ has interface $\{a_k, a_1\}$ and empty extension; its conditional degrees are all 1, so $\mu_\infty = 1$ and $\mu_1 = N_k$. With $p = \infty$ and unit weights this gives $B_{\max} = N_1 \prod_{j=2}^{k-1} M_j$.

**A feasible point for every common $p \geq 1$.** Put weight $\frac{p}{p+1}$ with exponent $p$ on every middle step, $\frac{p}{p+1}$ on step 1 and $\frac{1}{p+1}$ on the closing step with exponent 1. The cover constraints read: $a_1$: $\frac{p}{p+1} + \frac{1}{p+1} = 1$; $a_2$: $\frac{p}{p+1} + \frac{p/(p+1)}{p} = 1$; $a_i$ for $3 \leq i \leq k-1$: $\frac{p}{p+1} + \frac{1}{p+1} = 1$; $a_k$: $\frac{p}{p+1} + \frac{1}{p+1} = 1$. All are tight, so the LP optimum satisfies

$$B_{\mathrm{LP}} \leq N_1^{\frac{p}{p+1}} N_k^{\frac{1}{p+1}} \prod_{j=2}^{k-1} \mu_p(R_j)^{\frac{p}{p+1}} \quad \text{and} \quad \frac{B_{\max}}{B_{\mathrm{LP}}} \geq \Big(\frac{N_1}{N_k}\Big)^{\frac{1}{p+1}} \prod_{j=2}^{k-1} \frac{M_j}{\mu_p(R_j)^{p/(p+1)}}.$$

**The exponent must be common (or non-increasing).** The constraint of the attribute between middle edges $j$ and $j{+}1$ is $\frac{p_j}{p_j+1} + \frac{1}{p_{j+1}+1} \geq 1$, i.e. $p_{j+1} \leq p_j$. Choosing each edge's best exponent independently is therefore not feasible in general (e.g. $p_j = 1$, $p_{j+1} = \infty$ covers that attribute only to $\frac{1}{2}$); the LP may still mix exponents within a step.

**Growth.** At $p = 2$ a middle factor $M_j/\mu_2(R_j)^{2/3}$ exceeds 1 exactly when $\mu_2(R_j)^2 < M_j^3$: the sum of squared degrees is below the cube of the maximum degree. A few high-degree values satisfy it; many low-degree ones need not ($n$ values of uniform degree $d$ qualify iff $n < d$, and $10^4$ values of degree 1 plus one of degree 20 do not). When every middle factor is at least $c > 1$ the ratio grows geometrically in the number $k-2$ of middle edges. As $p \to \infty$ every factor tends to 1, and LpH-CEG returns $\min(B_{\max}, B_{\mathrm{LP}})$, so the realized ratio is always at least 1. The boundary factor $(N_1/N_k)^{1/(p+1)}$ is fixed when the boundary relation sizes are fixed as we vary $k$; it is 1 for a uniform cycle.

**Acyclic queries.** On a star the cover constraints force unit weight on every step and the LP returns the max-degree bound. On a path the LP can use an interface at $p = 1$ in place of an extension (on a path of three relations it can use weights $(1, 0, 1)$), so acyclicity alone does not force unit weights; what redundant coverage on cycles adds is the slack exploited above.

## 3 A spectral reading of the common-neighbour statistic (post hoc)

For an even $k$-cycle of binary relations the count is a trace, $|\mathcal{Q}| = \mathrm{Tr}(A_1 \cdots A_k)$ for the incidence matrices $A_i$ ($v_i \times v_{i+1}$), and the common-neighbour $\ell_2$ norm is the Frobenius (Schatten-2) norm of the block product $B = A_i A_{i+1}$. Hölder's inequality for Schatten norms gives, over $m = k/2$ consecutive blocks, $|\mathcal{Q}| \leq \prod_j \|B_j\|_{S_m}$, and $\|B\|_{S_m}^m \leq \sigma_{\max}(B)^{m-2} \|B\|_F^2$ with $\sigma_{\max}$ bounded above by a Collatz–Wielandt certificate on $B^\top B$. Two numbers per block, both from the pair join ($\sigma_{\max}$ lies outside the polymatroid catalog), give a bound tighter than the polymatroid program on 96/240 long-suite queries (by a median 1.25× there, though 1.11× looser over all) and never below an available exact count; with each block's top 24 singular values (Lanczos; uncertified, yet above every exact count) it is tighter on 220/240 and takes IMDb at $|V|$=10 from 14.9× to 3.2× the truth. The next step on the statistics axis may thus be spectral order rather than block size.

## 4 Which statistics are new: a vocabulary table and a separation

For a binary block $R(a,b) \bowtie S(b,c)$ the statistics the paper uses or compares against are the following ($\deg_R(b)$ is the number of $a$ with $(a,b) \in R$, and $N_R(a)$ the set of $b$ with $(a,b) \in R$).

| Statistic | Definition | Pair join? | Used by |
|---|---|---|---|
| single-relation norms | $\|\deg_R(\cdot \mid a)\|_p$, $\|\deg_R(\cdot \mid b)\|_p$ | no | LpBound, all programs |
| pair-join size | $\sum_b \deg_R(b) \deg_S(b)$ | no | CorrBound ($\ell_1$ inner product) |
| shared-attribute products | $\sum_b \deg_R(b)^p \deg_S(b)^q$ | no | CorrBound's generalized inner products (Lemma 3.1) |
| one-side interface norms | $\big\|\big(\sum_{b \in N_R(a)} \deg_S(b)\big)_a\big\|_p$ | no | this paper (join-free) |
| common-neighbour norms | $\big\|\big(\lvert N_R(a) \cap N_S(c)\rvert\big)_{a,c}\big\|_p$ | yes | this paper |
| claw pairs (bi-stars) | $\sum_{(a,b) \in R} \deg_R(a)^p \deg_R(b)^q$ | no | Lin–Wang (arXiv:2510.04249) |
| caterpillars | tree homomorphisms into one $R$ | no | Lin–Wang (arXiv:2604.14647) |

**Proposition 2** (Separation)**.** *The common-neighbour $\ell_2$ norm is not determined by the other rows of the table: there are two databases that agree on every single-relation degree sequence, on the pair-join size, on every shared-attribute product $\sum_b \deg_R(b)^p \deg_S(b)^q$ ($p, q \geq 0$) and on every one-side interface norm, yet differ in the squared common-neighbour $\ell_2$ norm (*32 *against* 24*), and so in the number of* 4*-cycles they close.*

*Proof.* Take $R$ with the $4 \times 4$ incidence matrix $M$ and $S$ with $M^\top$, for $M = A$, two disjoint copies of $K_{2,2}$, and $M = B$, the bipartite 8-cycle (row $i$ adjacent to columns $i$ and $i{+}1 \bmod 4$). Both are 2-regular, so every degree in $R$ and $S$ is 2, every shared-attribute product is $4 \cdot 2^{p+q}$, the pair-join size is 16, and every one-side interface degree is 4. The common-neighbour matrix is $MM^\top$: for $A$ it has eight entries equal to 2 (squared norm 32); for $B$, four diagonal entries 2 and eight entries 1 (squared norm 24); the largest common-neighbour count is 2 in both. For the 4-cycle $R(a,b) \bowtie S(b,c) \bowtie R'(c,d) \bowtie S'(d,a)$ with $R' = R$, $S' = S$ as separate atoms, the count is $\mathrm{tr}((MM^\top)^2) = \|MM^\top\|_F^2$, again 32 against 24. □

The two databases also agree on the single-relation statistics of Lin and Wang: every relation is 2-regular on $4 + 4$ vertices, so the number of homomorphisms of any tree with $m$ edges into it (a star, a claw pair, a caterpillar), with its root on either side, is $4 \cdot 2^m$ in both. Richer single-relation statistics therefore cannot recover the common-neighbour norm either; the two families are complementary, and we have not evaluated their bounds (their inference differs from the programs here).

The separation is limited to the statistics listed: it does not claim that no multi-relation term of CorrBound, or any other statistic, determines the common-neighbour norm, and it says nothing about the size of the gain on real data (the paper's ablations measure that).

# 5 Which statistic needs the join: three-norm details, sketch and selection

Of the eight interfaces of a block of binary relations $R(a,b), S(b,c)$, seven (five non-trivial) come from a scan of one relation with degree lookups in the other (the degree of $a$ is $\sum_{b \in N_R(a)} \deg_S(b)$, that of $b$ is $\deg_R(b) \deg_S(b)$); only $\{a,c\}$, the common-neighbour count $|N_R(a) \cap N_S(c)|$, needs the pair join; its $\ell_2$ is determined neither by the other interfaces nor by shared-attribute inner products (Section on the separating example), while its $\ell_1$ is the join size. We compute the seven without the join (identical to the materialized statistics on 120/120 Hetionet queries; 0.49 s against 4.6 s per block). With three norms and the flow LP they stop a median 2.3× (max 18.7×) above the full $bs$=2 bound on the Hetionet long suite (1.5–2.9× on the families, 1.9–57× on the sweep), and the common-neighbour $\ell_2$, $\|A_R A_S\|_F$ for $A_R \in \mathbb{R}^{|a| \times |b|}$ and $A_S \in \mathbb{R}^{|b| \times |c|}$, closes the gap on 12/12 family–seed pairs, 10/10 sweep points, 85/120 Hetionet and 116/120 IMDb queries (Hetionet median 1.00×, max 1.67×); its $\ell_\infty$ does not (1.70×), and added to $bs$=1 alone it reaches 1.03× (53/120 exact), so the attribution is order-dependent. In the headline setting (eleven norms, polymatroid) the join-free seven carry a median 88% and 86% of the log-gain on Hetionet and IMDb, the $\ell_2$ leaves 1.47× and 1.39× (full on 1/120 each), and the other common-neighbour norms (with the join-free $\ell_1$) close the rest. A one-pass triangle-inequality bound on the $\ell_2$ recovers nothing on Hetionet (2.2×); a 64-probe Hutchinson sketch lands within 0.91–1.78× of the full bound in 0.32 s per block but is an estimate, not a bound. Post hoc, it picks which joins to pay for: joining the three of a $|V|$=6 query's six blocks whose sketched norm lowers the bound most recovers the full $bs \leq 2$ bound on the median of 34 Hetionet queries (top two on 32 IMDb ones), against half that gain for a random three; timed end to end, this plan takes a median 0.67× the time of joining every block on Hetionet

(33/40 cheaper) but 1.39× on IMDb, where join-free statistics, sketches and ranking cost more than the three joins they skip. These times are against materialized pair joins. The sparse-product path of the paper's cost–tightness table computes every common-neighbour norm without join tuples, for 8.2× and 8.3× LpBound's collection, so choosing which blocks to join matters only where the joins are materialized.

## 6 Which block size to pay for: cost-study details

At $bs$=1, collecting degree sequences on the four cost-study families costs 22–27 ms against ∼1.9 ms to solve; $bs$≥2 multiplies that term by a median 45× there (24× more entries) and 61× (8–558×) on the Hetionet suite, and a cataloged $bs$=2 estimate costs 10 ms (the paper's long-suite table). Use $bs$=1 when the bound is re-solved many times per statistics build and $bs$≥2 when the pair joins can be materialized ahead of the workload.

## 7 IMDb at five times the films, and the YAGO count gap

*IMDb at* 1,000 *films.* At five times the films of the paper's slice (1,000; the 40 queries at $|V|$=6, bounds only, three norms; exploratory) every tightness ratio grows: the inference gap from 3.8 to 4.8×, block statistics from 4.0 to 7.0× under the cover LP and from 17 to 48× under the flow LP, 40/40 below entropic, the program gap from 4.2 to 6.8×. Collecting the statistics grows faster than the data: 4.2× the rows cost 48 s per query at the median (from 4.4 s), up to 25 min and 16 GiB, measured with 10 workers sharing the machine.

*YAGO counts.* Our exact counts differ from G-CARE's on 49 of the 192 YAGO queries we evaluate. A second implementation of the same semantics that reads G-CARE's raw files without DuckDB reproduces our count on all 49 of them, and 45 of them lie in G-CARE's Petal families (45 of 100 Petal queries, against 4 of 92 others), so the gap is in semantics or preprocessing we could not reconstruct, not settled either way. We then tested the semantic choices one at a time (`scripts/gcare_semantics/`, outputs in `results/gcare/semantics/`). G-CARE defines the count as a directed homomorphism with label-subset vertex matching, and its loader reads one label per query vertex and all labels per data vertex, which is our rule. The YAGO file has G-CARE's published statistics (12.8M vertices, 15.8M edges, 91 edge labels). Vertex labels cannot matter: every disputed query has only wildcard vertices. Neither undirected nor injective matching reproduces any disputed count, and both break agreeing controls (G-CARE's counts on queries where the two semantics differ are homomorphism counts); counting duplicate edges changes nothing; no reversal of any subset of query edges (512 variants) and no single edge-label substitution (810 variants) reproduces the four and two Petal queries tried. The cause therefore lies outside the stated semantics (how the published Petal counts were computed is not documented), and we keep the 49 queries separate rather than resolve them.

## 8 Degree sequences from catalog histograms

Degree sequences can come from catalog histograms at `ANALYZE` time: over buckets $(d_i, c_i)$ (degree $d_i$ held by $c_i$ values), $\|\mathbf{d}\|_p = (\sum_i c_i d_i^p)^{1/p}$. We have not evaluated this.

# 9 CorrBound's cyclic workload

CorrBound's public artifact (`https://github.com/christophmayerm/CorrBound`, MIT; inspected revision `8e05eb7`) ships the workload of its cyclic experiment (query graphs from Sun and Luo's subgraph-matching study, in the SQL form and with the truths of LpBound's code base): 400 labelled subgraph-matching queries with 8 vertices (200 dense, 200 sparse; 11–28 edges) on DBLP (317K vertices, 2.1M directed edge rows, symmetric), its estimates for CorrBound-e (exact degree vectors) and LpBound (flow bound), and true cardinalities. Each edge atom becomes a binary relation over its two query vertices holding the edges whose endpoints carry the query's labels; the vertex atoms are key lookups with a label predicate, and CorrBound's own LP drops them too. The initial comparison imports CorrBound-e's and LpBound's estimates from the artifact's result files; the re-runs described below independently check those imported values.

*Same norms, same baseline.* CorrBound's LpBound statistics use $p \in \{1, \dots, 30, \infty\}$, and so do we. Our single-relation polymatroid bound reproduces the artifact's LpBound estimate on 400/400 queries (ratio 1.000000 at both extremes; the artifact computes it with the flow LP and we with the polymatroid program, which coincide for single-relation degree constraints), so both sides start from the same statistics and the same baseline.

*Results.* Block statistics ($bs \leq 2$, every interface) under the polymatroid program sit a median 1.99× above the exact count (P95 3.9×), against 6.1× (P95 20×) for CorrBound-e and 64.6× (P95 190×) for LpBound. They are tighter than CorrBound-e on 382/400 queries (191 of each kind), a median 3.0×, and at most 1.51× looser on the other 18. No bound of any method falls below the exact count.

*The counts.* The artifact's true cardinalities sit below the count its own SQL returns on its own data on all 400 queries (a median 0.6%, at most 39%); we checked three queries by running the artifact's SQL verbatim, and requiring distinct images for same-label vertices does not reproduce them. We therefore check validity against a recount with DuckDB over the same relations; against the artifact's counts no bound falls below either.

*What each side has.* CorrBound-e adds, for every query vertex with $k$ incident edges, the $k$-way generalized inner products of their degree vectors ($p = 1, 2, 3$) and intra-relation correlation norms; our $bs \leq 2$ catalog has neither. It has the interface-conditioned norms of every pair block. The released cyclic implementation does not explicitly collect those block-interface norms. Proposition 2 separates the common-neighbour norm only from the listed pairwise statistics, not from CorrBound's general Lemma 3.1. CorrBound's statistics are computed once per dataset; ours cost a median 25 s per query in our Python prototype (all pair blocks, 31 norms, materialized joins), and the solve 0.38 s.

*Re-running CorrBound-e.* We ran the artifact's own LP builder (`corrbound.cyclic.LPBuilder`, OR-Tools GLOP; its flow-LP extension, unused by CorrBound-e, stubbed) on statistics we computed in its formats from its DBLP files (`scripts/corrbound_sm_artifact_lp.py`). Run unchanged it reproduces the published estimate on 400/400 queries (ratio 1.000000 at both extremes). Its inter-relation inequality is written $p \sum_i h(xy_i) - (k-1)h(x) \leq \log_2 \sum_v \prod_i d_i(v)^p$ and its intra-relation one $(p+q+1)h(AB) - h(A) - h(B) \leq \log_2 \sum_{(s,t)} d(s)^p d(t)^q$, for every $p \in \{1, 2, 3\}$ and every $p, q \leq 10$ with $pq \leq 12$ respectively. The paper's Eq. (13) and (14), $h(x) + p \sum_i h(y_i \mid x)$ and $h(AB) + p\,h(B \mid A) + q\,h(A \mid B)$ on the left, give $-(pk-1)h(x)$ and $-p\,h(A) - q\,h(B)$; the two agree at $p{=}q{=}1$. For larger $p$ the coded forms are not implied by the Lemma: on a complete bipartite relation $K_{m,m}$ the uniform distribution makes the coded intra-relation left side $8 \log_2 m$ at $p{=}q{=}2$, against a right side of $6 \log_2 m$. With Eq. (13)/(14) and nothing else changed, the builder's estimates are a median 2.4× (1.5–6.1×) higher, 15.2× above the exact count; neither version falls below an exact count on these queries. The paper's comparison uses the published estimates, which

is the conservative comparison for block statistics; against the Eq. (13)/(14) version, block statistics are a median 7.3× tighter. We reported the coefficient difference to the authors in the artifact's issue tracker.

*Combined catalog.* Adding CorrBound-e's inter- and intra-relation inequalities (Eq. (13)/(14)) to our polymatroid program over the $bs{\leq}2$ block catalog (`scripts/corrbound_sm_combined.py`; the program gets `polymatroid_bound`'s `extra` constraints) tightens 28 of the 400 queries, by at most 3.1% (median over the exact count 1.985× against 1.987×). On this workload the block statistics carry nearly all of what CorrBound-e's statistics add. In the same program, CorrBound-e's statistics alone give 15.1× over the exact count, against 15.2× in the artifact's LP, whose Shannon cone omits the $h(x) + h(y) \geq h(xy)$ inequalities.

## 10 Additional interpretation of comparison results

The following details complement the main comparisons. *Pair-join sizes on IMDb* (the paper's IMDb paragraph keeps the self-join explanation): where half the pairs are self-joins, sizes add almost nothing over LpBound, while the interface-conditioned statistics add 17–171×. *Collection cost:* the 61× overhead of blocks, growing faster than the data, calls for cataloging inside the system rather than per-query collection. *Join evaluation:* Yannakakis+ makes Yannakakis-style evaluation of acyclic queries practical in DuckDB and PostgreSQL, the complement of our cyclic setting; CompactLTJ shrinks Leapfrog Triejoin; Capelli et al.'s sampling variant takes tree-superadditive prefix upper bounds, which our residuals could supply if they qualify, though in our 66-query study they do not beat max-degree ordering in general.

## 11 Intermediate norms on an acyclic path

Theorem 4 (cyclic improvement) is a sufficient condition. Intermediate norms can also tighten the cover bound on a path. Take $Q = R(a,b) \bowtie S(b,c) \bowtie T(c,d)$ with $R = \{(a_0, b_0)\}$, $S = \{(b_0, c_i) : 0 \leq i < 10\}$ and $T = \{(c_0, d_j) : 0 \leq j < 100\} \cup \{(c_i, d_0) : 1 \leq i \leq 900\}$, so $|R|, |S|, |T| = 1, 10, 1{,}000$ and $|Q| = 109$. On the hyperpath $R, S, T$ with $bs{=}1$, the cover LP gives 1,000 with $\mathcal{P} = \{1, \infty\}$ and $\sqrt{10} \cdot \sqrt{10{,}900} \approx 330.15$ with $\mathcal{P} = \{1, 2, \infty\}$: weights $(1, \frac{1}{2}, 1)$ with exponents $(1, 1, 2)$ cover the attributes $(a, b, c, d)$ at $(1, 1.5, 1, 1)$, and the $\ell_2$ of $T$'s $c$-degrees ($\sqrt{100^2 + 900}$) replaces part of $S$'s extension. The cardinalities are the same on both sides; only $\ell_2$ is added. A second instance (with $|S| = 22$, $|T| = 100$) goes from 100 to $\sqrt{20 \cdot 190} \approx 61.64$. Both values are what the released estimator returns, and LpBound's entropic bound on the same statistics equals them. Our experiments do not measure how often this happens on acyclic workloads, and the stars we ran gain nothing (each leaf is covered by one step only).

## 12 Independent data slices

Every query of a suite reads the same slice, so the query counts are not independent data conditions. `scripts/slice_independent.py` reruns the headline comparison (LpBound's entropic bound against the $bs{\leq}2$ polymatroid bound, eleven norms on both sides, pair statistics from the sparse collector) on slices fixed before any result was seen: for Hetionet, ten uniform 20K-row samples of every full file (seeds 1–10, one per slice, the same sample of a relation in every query), and for IMDb six further 200-film sets (`prepare_imdb_cycles.py --rank-offset` and `--random-from`): four lower popularity strata and two uniform draws from the 5,000 most-voted films. All 120 registered queries of a suite run on each of its slices, with exact counts under set semantics where

DuckDB finishes in 60 s. The registered slices reproduce the stored headline exactly. On every slice every query's block bound lies below LpBound's (120/120), and no bound falls below an exact count (114–115 checked queries per Hetionet slice, 105–117 per IMDb slice).

Table 1: Median polymatroid bound on block statistics below LpBound, by slice (eleven norms on both sides; last column: queries with an exact count).

| Slice | $\|V\|$=6 | 8 | 10 | all | exact |
|---|---|---|---|---|---|
| Hetionet prefix (registered) | 236× | 1,724× | 10,160× | 1,724× | 115 |
| Hetionet, 10 samples (range) | 233–244× | 1,702–1,811× | 10,040–10,930× | 1,702–1,811× | 114–115 |
| IMDb top 200 (registered) | 9.74× | 29.9× | 86.8× | 29.4× | 105 |
| ranks 201–400 | 8.05× | 27.7× | 91.8× | 27.7× | 107 |
| ranks 401–600 | 8.34× | 31.8× | 90.2× | 30.3× | 109 |
| ranks 801–1,000 | 9.59× | 33.4× | 125× | 32.7× | 113 |
| ranks 1,601–1,800 | 7.83× | 34.8× | 102× | 32.7× | 117 |
| random of top 5,000, s1 | 9.64× | 33.8× | 113× | 32.3× | 117 |
| random of top 5,000, s2 | 8.07× | 25.9× | 87.9× | 26.7× | 117 |

The margins move little across slices of the same source: within 0.99–1.05× of the registered prefix on Hetionet and 0.91–1.11× of the registered film set on IMDb, overall. This bounds the dependence on the row-order prefix and on film popularity; it does not make the slices independent databases from other sources (the samples share their source files), and other graphs or schemas may behave differently. An earlier check, on the $|V|$=6 queries only and with each relation replaced by a uniform 20K-row sample of the full file (two fixed seeds) the $|V|$=6 medians are 46/46/49× (cover) and 236/233/244× (polymatroid) for prefix/seed 1/seed 2, all 40 below LpBound in each. A 30K-row prefix (a larger relation in 39 of the 40 queries) keeps the cover margin at 46× (40/40).

## 13 G-CARE YAGO by topology

The YAGO set we evaluate is the part of G-CARE's cyclic queries that fits the prototype's size caps, so it is not a random subset, and our counts differ from G-CARE's on part of it. Table 2 breaks it down (`scripts/gcare_topology.py`); gain is LpBound's entropic bound over the polymatroid bound on block statistics, eleven norms. The 143 agreeing queries are the external check proper; the 49 others are our reconstruction of G-CARE's semantics. Inclusion depends on topology (Cycle_6: 25 of 90; Petal_6: 50 of 78), and so does the gain, so the overall median describes this subset, not YAGO's cyclic workload. On AIDS all 241 queries run and agree (median 15.2×).

## 14 Bootstrap intervals of the gains

The paper reports that query-level bootstrap intervals keep each median ratio to LpBound on one side of 1. For the polymatroid program on block statistics that side is guaranteed by construction (its constraints contain LpBound's), so the informative quantity is the size of the gain. Table 3 gives it: median gain over LpBound (eleven norms) with its 95% interval, from 10,000 resamples of the registered queries of the fixed databases (`results/suite/bootstrap_ci.csv`). These are conditional query-bootstrap intervals: resampling treats the registered queries as exchangeable units. Queries share relations and motifs, so the intervals are descriptive resampling sensitivity, not uncertainty across independent databases. They do not cover other snapshots, row samples or pilot choices, and are pointwise rather than simultaneous across comparisons.

Table 2: YAGO cyclic queries by topology: G-CARE's set, evaluated, count equal to G-CARE's, and median gain (all / agreeing / not).

| Topology | cyclic | run | agree | gain | agree | not |
|---|---|---|---|---|---|---|
| Clique_10 | 8 | 8 | 7 | 2.84× | 2.84× | 71.2× |
| Clique_6 | 1 | 1 | 1 | 1.54× | 1.54× | – |
| Cycle_3 | 57 | 41 | 41 | 2.92× | 2.92× | – |
| Cycle_6 | 90 | 25 | 25 | 68.9× | 68.9× | – |
| Cycle_9 | 12 | 3 | 3 | 5,930× | 5,930× | – |
| Flower_6 | 12 | 10 | 10 | 8.67× | 8.67× | – |
| Graph_12 | 2 | 0 | 0 | – | – | – |
| Graph_3 | 2 | 2 | 0 | 168× | – | 168× |
| Graph_6 | 10 | 2 | 1 | 161× | 316× | 6.69× |
| Graph_9 | 10 | 0 | 0 | – | – | – |
| Petal_12 | 22 | 13 | 1 | 1,213× | 1,213× | 1,427× |
| Petal_6 | 78 | 50 | 50 | 31.3× | 31.3× | – |
| Petal_9 | 62 | 37 | 4 | 87.5× | 39.0× | 93.2× |
| all | 366 | 192 | 143 | 24.4× | 12.1× | 311× |

Table 3: Median gain over LpBound with 95% bootstrap interval.

| Suite | $\|V\|$ | cover $bs{\leq}2$ | interval | polymatroid | interval |
|---|---|---|---|---|---|
| Hetionet | 6 | 45.7× | [31.1, 69.3] | 236× | [155, 335] |
| Hetionet | 8 | 269× | [181, 337] | 1,724× | [1,213, 2,567] |
| Hetionet | 10 | 920× | [767, 1,204] | 10,160× | [7,831, 13,077] |
| IMDb | 6 | 1.27× | [1.14, 1.34] | 9.73× | [8.25, 12.2] |
| IMDb | 8 | 2.88× | [2.58, 3.33] | 29.8× | [24.8, 34.5] |
| IMDb | 10 | 6.83× | [5.97, 7.94] | 86.8× | [71.1, 96.5] |

## 15 Statistics outside a proxy for CorrBound's

The interfaces of a block of two binary relations $R(a,b), S(b,c)$ fall into four groups by what they count: the pair-join size ($\emptyset$); functions of $\deg_R(b)$ and $\deg_S(b)$ alone ($\{b\}, \{a,b\}, \{b,c\}, \{a,b,c\}$), whose $\ell_p$ norms are the inner products $\sum_b \deg_R(b)^q \deg_S(b)^r$ with $(q,r) \in \{(1,1),(p,p),(1,p),(p,1)\}$, a subset of CorrBound's (its Lemma 3.1 allows a separate $p_i$ per relation and $k$-way products); the one-side interfaces $\{a\}, \{c\}$; and the common-neighbour interface $\{a,c\}$. Table 4 adds them in that order to LpBound's single-relation statistics (eleven norms, polymatroid program, median gain over LpBound over 120 queries per suite; `scripts/ablation_overlap.py`). The proxy is exact (no sketches) but covers only these exponent pairs and no $k$-way products, so the shares below are relative to it. Beyond it, the one-side and common-neighbour statistics tighten the bound on 120/120 Hetionet and 120/120 IMDb queries, a median 34.4× and 13.9×, and carry a median 48% and 79% of the log-gain; on their own they come within 1.03× and 1.03× of the full catalog's median.

## 16 Leave-one-out and add-one over the statistic groups

The cumulative table above adds the groups in one order, and the three-norm ablation another; an attribution read off one order can depend on it. `scripts/ablation_loo.py` therefore removes each group from the full catalog and adds each group alone to LpBound's single-relation statistics, in

Table 4: Cumulative catalogs: median gain over LpBound.

| Catalog | Hetionet | IMDb |
|---|---|---|
| LpBound (single relation) | 1× | 1× |
| + pair sizes | 15.1× | 1.00× |
| + every pairwise product | 41.8× | 1.97× |
| + one-side | 608× | 18.4× |
| + common-neighbour (full) | 1,724× | 29.4× |
| one-side + common-nbr only | 1,674× | 28.7× |

the headline setting (eleven norms, polymatroid program, pair statistics from the sparse collector that the cost table times; 120/120 queries per suite finish). Table 5 gives the median ratio to the full catalog's bound (the full catalog is a median 1,724× and 29.4× below LpBound) and how many queries are within 1.0001× of it.

Table 5: Leave-one-out and add-one: median bound over the full catalog's (queries within 1.0001× of full, of 120).

| Catalog | Hetionet | IMDb |
|---|---|---|
| full − pair sizes | 1.00× (120) | 1.00× (120) |
| full − shared products | 1.00× (98) | 1.00× (85) |
| full − one-side | 1.79× (0) | 3.82× (0) |
| full − common-nbr | 2.56× (1) | 1.56× (1) |
| full, common-nbr $\ell_2$ only | 1.47× (1) | 1.39× (1) |
| LpBound + pair sizes | 105× (0) | 29.4× (0) |
| LpBound + shared products | 34.4× (0) | 13.9× (0) |
| LpBound + one-side | 3.08× (1) | 1.56× (0) |
| LpBound + common-nbr | 1.93× (0) | 6.25× (0) |
| LpBound + common-nbr $\ell_2$ | 2.08× (0) | 8.72× (0) |

The pair sizes are implied by the other groups on every query, and the shared products matter on only 22 and 35 queries. The one-side interfaces and the common-neighbour norms are each needed, and neither suffices alone, with opposite weights: on Hetionet removing the common-neighbour norms costs more (2.56× against 1.79×), on IMDb removing the one-side interfaces does (3.82× against 1.56×), and the common-neighbour $\ell_2$ alone, added to LpBound's statistics, recovers a median 90% of the log-gain on Hetionet but 35% on IMDb. The statement that the common-neighbour $\ell_2$ carries the gain therefore holds for the three-norm flow-LP ablation and for Hetionet; at eleven norms on IMDb the join-free one-side interfaces carry most of it.

## 17 Join ordering on IMDb with the headline estimator

`scripts/imdb_plan_final.py` chooses, for each of the 40 IMDb cycles at $|V|$=6, the connected left-deep order minimising $C_{\text{out}}$ under LpBound's bound (eleven norms, its flow LP), the released guarantee `guaranteed_block_bound` over the $bs \leq 2$ catalog (eleven norms, sparse collector; the polymatroid optimum at these sizes) and exact prefix counts, and times the chosen orders, 16 sampled ones and DuckDB's own plan five times each, one query at a time, each repetition in a shuffled order; an order that exceeds 60 s in the first repetition is kept at the budget and not repeated (the server was shared with other users, median 1-minute load 12.8). On the 33 queries

whose forced orders all finish, the median slowdowns over the best timed order are 1.05× (block bound), 1.08× (LpBound) and 1.06× (exact counts), P95 2.08, 2.26 and 1.34×, and DuckDB's plan 0.26× (P95 1.16×); paired, the block bound's pick is faster than LpBound's on 6 queries, slower on 3 and within 5% on 24, and the per-order timing CV is a median 6.5–7.4%. On the other 7 queries every strategy's pick, or nearly every sampled order, exceeds 60 s while DuckDB finishes, so excluding them favors the forced orders. Planning time, the statistics and solves for every prefix a strategy needs, is a median 12.5 s for the block bound and 2.2 s for LpBound; with it, the block bound's plan-plus-execution time is a median 824× DuckDB's (161× for LpBound). The block statistics thus make the left-deep choice slightly better on this workload but do not pay for themselves here; a workload-level catalog collected once (as in the paper's catalog experiment) would remove most of the planning time but not DuckDB's advantage from bushy plans.

*The earlier protocol.* `scripts/imdb_plan_injection.py` ran the same protocol on the 40 IMDb cycles at $|V|$=6, with four estimators behind $C_{\text{out}}$, tests the thesis rather than the $bs$=1 bound. Block statistics move the choice toward the true-cardinality order: the flow LP on $bs$=2 (three norms, an uncertified candidate, not the headline's eleven-norm polymatroid estimator) picks the order exact counts would pick on 24/40 queries, and on the 33 whose forced orders all finish within 60 s its median slowdown over the best timed order (the strategies' picks plus 16 sampled, of the 96 connected left-deep orders) is 1.19×, against 1.41× for the $bs$=1 cover LP, 1.23× for LpBound (three norms) and 1.21× for exact counts—differences among the last three lack a repeat-based uncertainty estimate. None competes with DuckDB's optimizer here (0.43× median, at most 2.0×), and on the other 7 queries nearly every forced order exceeds 60 s while DuckDB finishes, so excluding them favors the forced orders. Each order was timed once, 10 queries concurrently, so equivalence is not established either. A better bound improves *which* left-deep plan is chosen, not whether one should be.

## 18 Tail robustness by shape, with every timing kept

`scripts/robustness_repeat_raw.py` repeats the paper's 24-query execution study with every observation written out (`results/e2e/robustness_raw.csv`: query, plan, run, position in the shuffled run, time). Each plan runs six times in shuffled order after a warm-up; the reference plan (fastest forced left-deep order) is chosen on runs 1–2 and measured on runs 3–6, so choosing and measuring do not share a run. Table 6 gives the slowdowns over that reference and the paired ratio of the bound-guided plan's time to DuckDB's. Percentiles use the sorted observation at zero-based index $\lfloor q(n-1) \rfloor$ (P95 is the third largest for $n = 24$). Slowdowns below one are possible because reference selection and evaluation use disjoint runs. The paired bound-guided/DuckDB ratio uses all six runs. The pooled P95 is 1.11× for the bound-guided order against 5.26× for DuckDB. The tail comes from the denser shapes: DuckDB is faster on triangles and 4-cycles and slower on diamonds and $K_4$, and the AGM filter that sets the suite depends on shape and size, so the pooled percentiles describe these four shapes, not a workload.

## 19 Cost measurement details

The script `cost_accuracy_indep.py` behind the paper's cost–tightness table runs every (query, catalog) pair as its own task in a fresh process (8 workers, shuffled order, one run), times collection (data loading excluded), the constraint build and the solve separately, and reads the process's peak RSS after loading and at the end; their difference is a lower bound on the extra memory. The sparse-product and materialized full catalogs give the same bound on all 240 queries (largest

Table 6: Slowdown over the reference (median/max, evaluation runs) and median bound-guided/DuckDB time ratio, by shape.

| | | bound-guided | | DuckDB | | ratio |
|---|---|---|---|---|---|---|
| Shape | $n$ | med | max | med | max | |
| triangle | 6 | 0.83 | 1.12 | 0.78 | 0.94 | 1.13 |
| 4-cycle | 6 | 1.02 | 1.10 | 0.59 | 0.70 | 1.78 |
| diamond | 6 | 1.04 | 1.75 | 2.39 | 2.81 | 0.53 |
| $K_4$ | 6 | 1.00 | 1.05 | 4.25 | 5.92 | 0.25 |
| all | 24 | 1.00 | 1.75 | 0.88 | 5.92 | 1.01 |

$|\Delta \log_2|$ 0). Peak memory over the loaded data is a median 60 against 626 MB on Hetionet and 44 against 262 MB on IMDb; the largest process peaks are 429 against 6,351 MB and 341 against 4,149 MB. Building the constraints takes about 2 ms. The workload catalog of the paper's Hetionet long suite holds 676 entries; stored as norms only, eleven per entry, it needs $676 \cdot 11 \cdot 8$ bytes, about 59 KB (degree sequences, kept by our prototype for norms not precomputed, are optional). An earlier version of the table measured the catalogs of a query in one process, in a fixed order, with the full catalog's joins materialized (`cost_accuracy.py`); its join-free and $\ell_2$ ratios (6.0/6.9× and 6.2/7.1×) are close to these, and its full-catalog ratio (52/24×) is the materialized path's.

*Repeats.* `scripts/cost_repeat.py` reruns the same 1,200 tasks three times, each repetition in its own shuffled order, with the BLAS and OpenMP pools pinned to one thread (8 workers; the server was shared with other users, at a median 1-minute load of about 21 on 40 hardware threads, recorded per task). Bounds agree exactly across repetitions; the per-query collection time varies by a median 3.8–4.9% across catalogs (coefficient of variation). The per-repetition medians of the collection ratio to LpBound are 7.8–8.2× (Hetionet) and 8.4–8.5× (IMDb) for the sparse full catalog and 48–52× and 23–24× for the materialized one; query-bootstrap 95% intervals of the median ratio (per-query median over repetitions, 2,000 resamples) are $[7.6, 8.6]$ and $[7.9, 9.1]$ (sparse) and $[35, 78]$ and $[20, 27]$ (materialized). The one-run values of the paper's table lie inside them. These are timing repeats on one database slice, not independent data conditions.

## 20 Candidate, certificate and returned bound by query size

For the synthetic cycle sweep of the paper's scalability section ($bs \leq 2$, three norms), Table 7 separates the three costs that the paper's flow-LP results combine: computing the polynomial-time candidate (all orders, total), certifying it by the $2^{|V|}$ polymatroid program, and the bound `guaranteed_block_bound` returns with its default limit $|V| \leq 12$ (the polymatroid optimum up to $k$=12, the cover LP beyond). The last column divides the returned bound by the flow candidate. At $k$=14 the certificate still finishes (1,425 s) and matches; at $k$=16 it does not finish in six hours, and the guaranteed value is the cover LP's, 288× above the uncertified candidate. The candidate's speed is therefore not a speed of the guaranteed estimator beyond $|V|$=12; there the returned bound is the cover LP's.

## 21 Sensitivity to the number of $p$-values

The paper's four-family sweep (cycle-4/6, $K_4$, chordal-5) of the cover LP's improvement as norms are added.

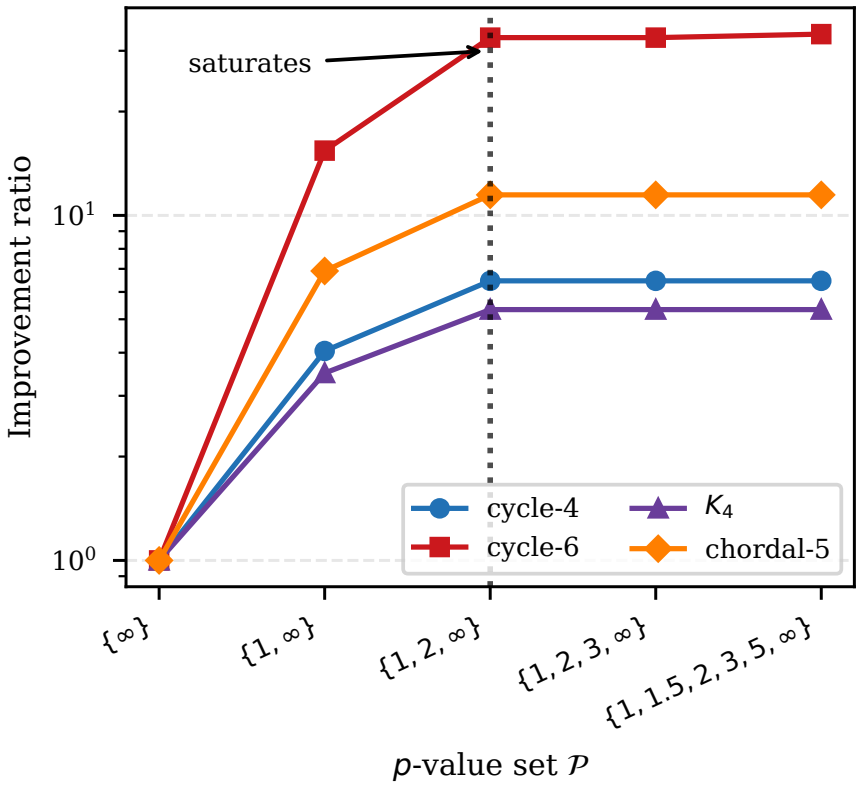



Figure 1: Sensitivity to the number of $p$-values across four query families (log scale). Improvement saturates at $\mathcal{P} = \{1, 2, \infty\}$ (dotted line) in every family; additional norms add $<2.4\%$.

Table 7: Flow candidate (ms), polymatroid certificate (s), candidate over optimum, returned method and returned bound over the candidate.

| $k$ | flow | poly | flow/poly | returned | ret./flow |
|---|---|---|---|---|---|
| 3 | 28 | 0.003 | 1.00× | polymatroid | 1.00× |
| 4 | 103 | 0.004 | 1.00× | polymatroid | 1.00× |
| 5 | 218 | 0.006 | 1.59× | polymatroid | 0.63× |
| 6 | 284 | 0.010 | 1.00× | polymatroid | 1.00× |
| 7 | 438 | 0.023 | 1.56× | polymatroid | 0.64× |
| 8 | 628 | 0.056 | 1.00× | polymatroid | 1.00× |
| 10 | 1,209 | 0.684 | 1.00× | polymatroid | 1.00× |
| 12 | 1,786 | 20.1 | 1.00× | polymatroid | 1.00× |
| 14 | 2,842 | 1,425 | 1.00× | cover | 136× |
| 16 | 4,438 | > 21,600 | – | cover | 288× |

## 22 Scope of the chain max-degree baseline

The experimental $\text{CEG}_M$ baseline uses `estimate_pessimistic` at $bs = 1$, restricting each step to a whole relation. Its DAG requires a nonempty interface after the first step; full $\text{CEG}_M$/MOLP permits additional attribute-subset transitions. For a checked binary example, let

$$\begin{aligned} R(a,b) &= \{(a_0,b_0),(a_1,b_0)\},\\ T(c,d) &= \{(c_0,d_j) : 0 \le j < 3\},\\ S(b,c) &= \{(b_0,c_0)\} \cup \{(b_i,c_j) : 1 \le i,j \le 4\}. \end{aligned}$$

The join count and full MOLP are 6, while the chain bound is 24. Thus the reported improvements over the chain are not general improvements over full MOLP. The comparisons against LpBound use the separate polymatroid baseline and are unaffected.

## 23 Full MOLP against the chain baseline

Full MOLP is the shortest path over all attribute subsets, with, for every binary relation $R(u,v)$, edges adding $\{u,v\}$ at cost $\log|R|$, $\{u\}$ at cost $\log|\pi_u R|$, and $v$ given $u$ at cost $\log \max_u \deg_R(v \mid u)$ (`scripts/molp_full.py`; on the counterexample to the chain's MOLP equivalence it gives 6 against the chain's 24). Table 8 sets it beside the chain on the paper's synthetic cycle table (the same ten seeds; means of per-seed ratios) and on the long suites (medians). On the synthetic cycles the chain is up to 237× looser than full MOLP: full MOLP covers alternate relations at their cardinality (on cycle-8, $800^4$, the AGM bound) or starts from projections, which the chain cannot. Against full MOLP the cover LP's gain is 1.4–5.1× and grows only about 1.35× per two added vertices (odd cycles, which an alternating cover does not fit, gain more). On Hetionet the chain is 1.2–2.9× looser than full MOLP; on IMDb the two coincide. The cover LP does not dominate MOLP: on Hetionet it is looser than full MOLP on 7 of 120 queries (by up to 2.4×). The comparisons against LpBound in the paper use the polymatroid program and are unaffected.

Table 8: Chain and full MOLP over the $bs$=1 cover LP, and chain over full MOLP.

| Instances | chain/cover | MOLP/cover | chain/MOLP |
|---|---|---|---|
| Cycle-3 | 2.75× | 2.75× | 1.00× |
| Cycle-4 | 8.43× | 1.44× | 5.86× |
| Cycle-5 | 23.8× | 3.73× | 6.40× |
| Cycle-6 | 68.9× | 1.89× | 36.6× |
| Cycle-7 | 211× | 5.09× | 41.6× |
| Cycle-8 | 614× | 2.60× | 237× |
| Hetionet $\|V\|$=6 | 1.85× | 1.39× | 1.18× |
| Hetionet $\|V\|$=8 | 5.06× | 2.07× | 2.13× |
| Hetionet $\|V\|$=10 | 8.89× | 3.60× | 2.93× |
| IMDb $\|V\|$=6 | 1.84× | 1.84× | 1.00× |
| IMDb $\|V\|$=8 | 1.91× | 1.91× | 1.00× |
| IMDb $\|V\|$=10 | 3.21× | 3.21× | 1.00× |

Table 9: Unified comparison of guaranteed upper bounds (synthetic power-law cycles, 100V/400E, one instance). "Cover" is our $bs$=1 LP; "Entropic" is LpBound's bound (via the $2^{|V|}$ program). Lower is tighter. "/CEG" is $\text{CEG}_M$'s ratio to Cover and "/Ent" Cover's ratio to the entropic optimum.

| Query | AGM | $\text{CEG}_M$ | Cover | Entropic | /CEG | /Ent |
|---|---|---|---|---|---|---|
| Cycle-3 | 8.0K | 15.6K | 6.8K | 5.0K | 2.3× | 1.36× |
| Cycle-4 | 160K | 655K | 116K | 87K | 5.7× | 1.32× |
| Cycle-5 | 3.2M | 25.5M | 1.97M | 1.45M | 13.0× | 1.36× |
| Cycle-6 | 64M | 1.12B | 31.5M | 25.2M | 35.7× | 1.25× |

# 24 Unified comparison on synthetic cycles

# 25 Additional $bs = 1$ experiments: JOB, TPC-H-like, data size and skew

On the JOB-medium subset (CI: 3K, MK: 2K, MC: 1K rows) the two co-actor 4-cycles improve 1.94× (with keywords) and 2.80× (with companies) at $bs = 1$ with weights $w$=0.5, and a star query by exactly 1.00×; the IMDb suite of the paper is the systematic version. On a small TPC-H-like synthetic schema (SF=0.01; not `dbgen` data) every cyclic query we built yields 1.00×; The sufficient condition in the cyclic theorem combines cyclic structure with degree skew in a middle relation; the acyclic counterexample above shows that cycles are not necessary. On synthetic cycle-4 (10 seeds) the improvement grows with data size, from 5.9±0.7× at 100 vertices to 33.9±1.5× at 2K (relative std <12%; a single-run sweep reaches 57× at 5K), and peaks at moderate skew: 2.3±0.5× at $\alpha$=0.5, 16.1±0.8× at $\alpha$=2.0, 6.6±0.4× at $\alpha$=3.0. The measured gain is smaller at both ends of this skew sweep. Low skew does not make the norms equal: $n$ equal degrees $d$ have $\ell_p$ norm $n^{1/p}d$. The theorem's factors, rather than equality of norms, determine whether an intermediate norm improves the bound.

Every number in this section is produced by the scripts in the repository and checked by its claim audit.

## 26 Multi-way ordering by per-step residuals

The per-step residuals can order a worst-case-optimal join greedily. As implemented, the residual of a variable prefix is the LP re-solved on the query with the prefix attributes projected out, which bounds the residual for every value of the prefix (and could be computed for LpBound in the same way): GenericJoin iterations drop 2.3× on the Hetionet cycle-4 in our simulator, but over 66 synthetic queries the bound-guided order is 1.6% worse at the median (20 wins, 43 losses; Wilcoxon $p$=0.003, indicative, as queries share shapes and seeds), so it is a targeted heuristic.

## 27 Complexity

Degree counts for one relation require a scan with grouping; the implementation also sorts the degree sequence. For a binary pair block, seven interfaces use relation scans and degree lookups, while the common-neighbour sequence is the nonzero entries of a sparse product $A_R A_S$. Its cost depends on sparsity and output size and need not be linear in the input. The earlier implementation materializes the pair join; both paths are retained for comparison. The cover LP has $m|\mathcal{P}|$ variables and $|V|$ constraints; it solves in milliseconds for the queries in the paper. The flow LP over a $bs \leq 2$ catalog is polynomial in the number of constraints and solves in 0.03–4.4 s at $k = 3$–16 over $4k$ orders.